Software, Hardware and Product Supply Chain Facility-Class Design Basis

The First Facility Class of the Information Technology Sector Design Basis

DBA-IT-SW-FC3 — Facility-Class Design Basis, DBA-IT Series

Issued Under: Information Technology Sector Framework (DBA-IT_Sector_Framework v0.4); DBA-MA Parent Framework

Version 0.2 — DRAFT

October 2026

Tim Roxey

Eclectic Technologies

Working Draft — Released for Review and Discussion

SUGGESTED CITATION

T. Roxey, "DBA-IT-SW-FC3 — Software, Hardware and Product Supply Chain," Version 0.2, in CIKR Design Basis Examples, Edition 1 (November 2026), Eclectic Technologies, November 2026. https://doi.org/10.5281/zenodo.23863986

This version is a member of CIKR Design Basis Examples, Edition 1 (November 2026) (https://doi.org/10.5281/zenodo.23863986), a dated edition that freezes, for each of fifteen critical infrastructure sectors, a sector framework, a facility class and a reference event as they stood on 1 November 2026. The edition is not falsified by a later revision of this document, whose working master continues to move. It is a working draft, marked DRAFT on its cover.

1. Purpose and Scope

1.1 Purpose

This document sets out the design basis for the software, hardware and product supply chain, the first facility class in the DBA-IT series. The *Information Technology Sector Framework* (DBA-IT_Sector_Framework) states the sector's design basis in general terms, and this document applies it to one function: the making and delivery of the products every other sector runs on. It names the events a product line must withstand, traces how each event reaches the systems that install the product and the facilities those systems serve, sets the parameters that size the class's protection, and states what the product line and the consuming facility must each be able to do when a trusted product turns out to carry a defect or an adversary. It describes no particular vendor or product. A design basis for a specific product line, vendor or consuming enterprise starts from this document and adds only what is particular to it.

The framework calls the software supply-chain compromise the logical-integrity consequence in its purest form: adversary capability inserted into a trusted product that then propagates to every system that installs it (framework §3.4). It leaves to the facility classes the development of the blast-radius consequence measure it proposes for the sector (framework §3.6, §9), and it names among its candidate bounding events a software supply-chain compromise inserted into a trusted update (framework §4.1). This document takes up both. On the author's decisions of 13 October 2026 the class is the sector's first, ahead of Connectivity and Protocols (DBA-IT-CP-FC2), because the six sectors that follow in the plan for closing the seven sectors will cite it; it is written for software products and their update channels in full, with the hardware and device supply chain stated at the depth of the sub-index for a later revision to develop (Section 2.2); and it proposes three reference events of its own (Section 9.2).

1.2 Scope

The class covers the production and delivery of software products and of the updates and content that change them, as the framework's register names them: operating systems, applications, security and industrial-control-system software, and the hardware and device supply (framework §3.4, §8.2). For software it covers the whole path from the source a developer writes to the code that runs on a consuming system: the source repository; the build system that turns source into an artifact; the signing that attests the artifact's origin; the distribution servers, mirrors and package registries from which it is fetched; the update channels by which it reaches installed systems, whether as versioned code or as content and configuration; the installation; and the privilege with which it then executes. It includes the third-party and open-source components a product carries, because a product delivers whatever is in it, and the people who maintain them.

Three neighbors are separate classes of this series. The cloud and platform layer, where a provider runs a service and its tenants consume it, is *Infrastructure and Compute* (DBA-IT-IC-FC1). The naming, routing and trust plane, including the certificate authorities that issue code-

signing certificates, is *Connectivity and Protocols* (DBA-IT-CP-FC2). The security operations centers, managed security service providers, incident response and the sector's information sharing are the coordination class, DBA-IT-OpsCo. They are covered here only where the product depends on them. Where a vendor signs its own artifacts with a certificate a public authority issued, the authority is the connectivity class's and the vendor's custody and use of the signing key are this class's; where a product reaches its hosts through a cloud console the vendor runs, the console is the compute class's and what the console delivers into the host is this class's.

Three boundaries with other series are drawn in the framework and are taken here as given (framework §1.2, §6.3). The physical compute on which builds run and from which updates are served is the data-center series'. The physical transport over which they travel is the communications series'. The fabrication of the semiconductors on which hardware rests is the critical-manufacturing series'. And every product this class delivers is consumed in a facility another class governs: the hospital's electronic health record is a product of this class running in a facility of *Care Delivery* (DBA-HPH-CD-FC1), and the engineering workstation of a safety instrumented system is a product of this class running in a facility of *Chemical Production and Synthesis* (DBA-CHEM-PR-FC1). The consuming class governs what the product does inside the facility. This class governs what the product brings into it. Section 8.3 states the boundaries.

What makes a product line a member of this class is the same at every vendor: it is trusted to change the systems that run it, and it changes many of them at once. The particular code, installed base and channel differ from product to product, and that detail belongs in each product line's own design basis.

1.3 The Reference Products

Three kinds of product are used as reference cases. Each was chosen because it shows a different way a trusted product can carry harm to the systems that accept it. In a physical class the reference cases are facilities; in this class they are positions, defined by what the product is trusted to do, how many systems trust it, and how fast it changes them.

The privileged infrastructure-management platform is installed inside an enterprise network to monitor and manage it, and to do so it holds broad credentials and reaches many of the enterprise's systems. It is updated on the vendor's release schedule, usually after the customer's own review. It bounds the integrity chain, because an adversary inside its build can deliver a capability that runs with the platform's reach in every enterprise that installs the update, and can choose which of them to use. The compromise of the SolarWinds Orion platform in 2020, conferred by the framework as DBA-IT-RE-1 (framework §9.1; Section 5.5), is the case.

The endpoint security agent runs inside the operating system's kernel on millions of hosts, because the visibility its purpose requires is available only there, and it is changed many times a day by content its vendor pushes to keep pace with threats. It bounds the availability chain, because a defect in what the vendor pushes reaches every host online in the window of release and can stop them all at the level from which they boot. The vendor's witness told the House

Committee on Homeland Security in September 2024 that the company released ten to twelve such content updates each day. The faulty content update of 19 July 2024, conferred by the framework as DBA-IT-RE-2 (framework §9.1; Section 5.1), is the case.

The upstream open-source component is a library maintained by one or a few people, often without pay, that operating-system distributions package and that privileged services link against. Its maintainers are trusted by everyone downstream and are known to almost no one. It bounds the dependency tier, because the component's reach is set by what links to it rather than by the size of its project, and because the people on whom its integrity rests can be cultivated, replaced or exhausted. The backdoor placed in the XZ Utils compression library in 2024, conferred by the framework as DBA-IT-RE-3 (framework §9.1; Section 5.6), is the case.

A fourth position is stated at the depth of the sub-index rather than as a reference case: the engineering software and field-device firmware that configure the control and protective functions of physical facilities. Its installed base is small and its reach into consequence is direct, because what it changes is the logic by which a plant is controlled and protected. It is where this class's logical consequence becomes a physical one, and Section 4.5 states the requirement at that boundary.

1.4 Class Type

This is a logical facility class under the author's determination of 30 September 2026 (Facility-Class Types Are Physical, Coordination and Logical), which types DBA-IT-SW-FC3 as logical: a system or service running on inherited physical infrastructure, whose consequence is a logical blast radius. In the framework's structure it sits under the Software and Product Supply Chain (SW) function class (framework §3.1, §3.4).

It is the first logical class written in the corpus. The physical classes before it each had a site, a plant and a population inside a fence, and their consequence ran through the plant. This class has none of those. Its product has no location of its own; its harm, when it comes, arrives in the facilities of other classes, often of other sectors; and a single act at one vendor reaches them all at once. Three things follow for the structure of this document. Its reference cases are positions rather than facilities (Section 1.3). Its parameters are stated in terms of reach, privilege and time rather than of mass, pressure or duration (Section 3.3). And the protective function on which its adequacy finally rests is not the class's own: it is the independence of the consuming facility's protective functions from what the class delivers (Section 6.7). The section order of the physical classes is kept so that the classes can be read across, and the content of each section is written for a product that runs everywhere and resides nowhere.

1.5 Relationship to Governing Documents

This instrument is issued under the Information Technology Sector Framework and inherits the DBA-MA parent framework. It does not repeat the framework's taxonomy, bounding principles, consequence measure or Bright Line and Command Broker architecture; it inherits them, and

Section 9 is the required inheritance map. Where the framework is still a skeleton (framework §4, §5 and §7), this instrument develops the class-level content ahead of it and records in Section 9 what the framework should take up. Section references of the form framework §N resolve against the Information Technology Sector Framework in force.

Governing instruments. This instrument does not own the concepts it applies. The Bright Line — its placement criterion, its granularity, its scope of application and the obligation it creates — is defined in FD-BL (*The Bright Line*), which governs. The Broker role that discharges that obligation is defined in FD-BR (*The Broker*). Layer decomposition, the determinism boundary as a scope gate, and the machine-learning carve-out are defined in FD-LD (*Layer Decomposition and Determinism*). Local sovereignty, the property of a protective function that executes independently of reachable digital authority, is defined in FD-LS (*Local Sovereignty*). Two determinations bind under FD-BL: FD-BL-D1 (*Mode of Discharge*) governs how the authorization obligation may be discharged, and FD-BL-D2 (*Indication Integrity*) governs the integrity of the basis on which an authorization is granted. FD-EV (*Envelope*) reserves the term envelope. Placement is on the consequence of the individual action, assessed per action; this instrument applies that definition and does not vary it, and where its application to this class is unsettled the instrument says so and refers the question (Section 7.3, Section 10.4). The Foundational Definitions are cited without a version token, and FD-REG (*Governing Stack Register*) is authoritative for their version state.

2. The Software and Product Supply Chain Facility Class

2.1 Defining Characteristics

Five characteristics define the class, and each is a reason a product can carry harm farther than any physical facility.

The first is trust by channel. A consuming system accepts what arrives over the product’s update channel because of where it came from, and the signature that proves it came from the vendor proves nothing about what it is. A trojanized update signed with the vendor’s own certificate passes every check of origin, and a defective update from the vendor’s own pipeline is genuine in every sense a signature can test. The class cannot take origin for integrity, and Section 4.2 states what follows.

The second is privilege. A product runs with the authority its purpose requires, and the products whose purpose is to manage, monitor or protect a system require the most: administrative credentials across a network, or execution inside the operating system’s kernel. Whatever the product delivers runs with that authority. A defect or an adversary in a product with kernel privilege is a defect or an adversary in the kernel.

The third is reach. A product is installed on every system that runs it, and the channel that changes it changes all of them. The reach of a single release is the product’s installed base, and

for the products the framework calls foundational it is a share of every sector's systems at once (framework §6.2).

The fourth is cadence. Products change continually, and the fastest-changing are those that must keep pace with an adversary: the content of a security product changes many times a day. Every change is an opportunity for a defect, and every routine change is cover for one that is not routine. The faster the cadence, the less any one change is examined.

The fifth is dependency. A product is built from components, many of them written by others and maintained by people the vendor does not employ. The product delivers its dependencies along with its own code, so the integrity of the product is the integrity of every component in it and of every person who can change one.

2.2 The Foundational-Dependency Sub-Index

The framework sub-indexes the sector's classes by foundational-dependency tier, so that the design-basis load a function carries reflects the breadth of the dependent population its compromise would reach (framework §3.1, §8.2). This instrument states three tiers for the class. A product's tier is set by its reach and its privilege together, and not by the size of its vendor or its project: a small library linked into a privileged service on most of the world's servers is in the first tier, and a large application that runs with a user's authority on a few systems is not.

The first tier is the foundational product. It runs with system or kernel privilege, or holds administrative reach across the networks it is installed in, and its installed base is a material share of the systems of more than one sector. Operating systems, endpoint security agents, the dominant infrastructure-management and remote-management platforms, and the widely linked open-source components on which they rest are in this tier. A defect or an adversary in a first-tier product is a cross-sector event of the highest tier the framework names (framework §6.2).

The second tier is the sector-critical product. Its installed base is concentrated in one sector, or it changes the logic by which physical processes are controlled and protected. The engineering software and firmware of control and safety systems, the clinical record and dispatch platforms on which hospitals and answering points run, and the core platforms of financial institutions are in this tier. Its reach is narrower than the first tier's and its consequence can be more direct, because it reaches a sector's facilities at the point where their own consequence is decided.

The third tier is every other product. It is in the class, and the class's provisions apply to it in proportion, but it does not carry the design-basis load of the first two.

The hardware and device supply chain carries the same tiers and is stated here at that depth only, on the author's decision of 13 October 2026. Network equipment, servers and their firmware, the devices people carry, and the field devices of control systems can each be counterfeited, tampered with in transit, or shipped with firmware that is itself a product of this class. The firmware is governed by the provisions written here for software. The physical device and its

interdiction are held for a later revision (Section 10.3), with the boundary to semiconductor fabrication in the critical-manufacturing series kept where the framework draws it.

2.3 Delivery Models and Range

The class spans several ways a product reaches its hosts, and the design basis turns on which a product uses. A vendor may push updates that install without the customer's action; it may publish updates that each customer stages and installs on its own schedule; or it may do both at once, versioned code under the customer's control and content or configuration on a channel of its own. An open-source component reaches its users by being fetched from a public registry at build time or by being packaged into an operating-system distribution, whose maintainers then become the next link. Embedded firmware may be updated over a network, by a field technician or not at all. And some consuming facilities, by policy, accept updates only on media carried across an air gap.

The range is wide. At one end is a component maintained by one person and linked into a large share of the world's servers; at the other a vendor whose agent runs on millions of hosts and whose channel delivers content to all of them many times a day. The class is written to fit both, because the consequence of either can reach a foundational share of the dependent population.

2.4 Consequence Context

The class's consequence is the logical blast radius the framework names: the breadth of the dependent population and the number of downstream sectors a compromise or failure reaches before it can be contained (framework §3.6). It takes three forms. An integrity consequence is adversary capability running inside the consuming systems with the product's privilege. An availability consequence is the consuming systems stopped or destroyed by what the product delivered. A confidentiality consequence is the information of the consuming systems taken through the product, which is often the integrity consequence's purpose. The forms combine: a capability delivered for espionage can be used for destruction, and an adversary who has the channel can deliver either.

None of the three needs a physical failure, and every one of them arrives in other sectors. On 19 July 2024 a single content update stopped about 8.5 million Windows hosts, on Microsoft's count. The Congressional Research Service recorded its consequence for 911 (IF12717): in Phoenix calls were answered and the callers' information recorded by hand, in Portland calls were answered manually, the Alaska State Troopers said 911 systems were not operating correctly, and police in Middletown, Ohio said 911 calls were not being received. At the House hearing that followed, the subcommittee's chairman spoke of hospitals that delayed or cancelled procedures, of banks' transaction processing, of federal agencies and of thousands of flights. In 2017 the NotPetya malware, delivered through the update channel of a Ukrainian accounting program, closed or slowed container terminals on five continents (*The NotPetya Attack on Maersk's Terminals*, DBA-TRAN-RE-1). The physical facilities in each case continued to run.

What stopped was the logical layer on which their operation depended, which is the framework's account of the sector exactly (framework §1.5).

3. The Facility-Class Design Envelope

3.1 Why the Supply Chain Is the First Class

Three reasons put this class first, and the first is the plan's. The plan for closing the seven sectors puts Information Technology first among them because nearly every other sector rests on it and its class will be cited by the six that follow; of the sector's three logical classes, the supply chain is the one every consuming sector meets directly, since every sector installs software and few operate a root name server or a hyperscale cloud. The second is the framework's: it names the supply-chain compromise as the logical-integrity consequence in its purest form and the sector's governing threat (framework §3.4, §5). The third is the record. The events of this class are among the best investigated in the sector, by the vendors themselves in published root-cause analyses, by the federal government in emergency directives and reports, and by the open-source community in the open, so the class's parameters can be stated against what is known rather than what is supposed.

3.2 The Two Consequence Chains

Every event in the class passes through the product's channel to the systems that trust it, and it does so by one of two chains.

Chain I, the integrity chain, begins with an adversary inside the product's path from source to host: in the source, in the build, in the signing, in the distribution, or in a dependency. The adversary's capability is delivered by the trusted channel, installed as the product, and executed with the product's privilege. It is latent until used, and its use is selective: the adversary chooses which of the systems that installed it to act on, and when. Its blast radius is the population that installed it, and its consequence is realized only in the part of that population the adversary chooses. The SolarWinds compromise is the case. The vendor notified about 33,000 customers and judged that fewer than 18,000 had installed an affected update (SolarWinds Corporation, Form 8-K, 14 December 2020), and nine federal agencies and about a hundred companies disclosed follow-on compromises, on the account of the National Counterintelligence and Security Center. The platform's reach bounds the campaign's consequence through this product, not the campaign's whole: the acting director of the Cybersecurity and Infrastructure Security Agency testified on 18 March 2021 that some victims' cloud compromises predated the backdoor, their initial vector unknown (*The Compromise of the SolarWinds Orion Platform*, DBA-IT-RE-1).

Chain A, the availability chain, begins with a defect in what the vendor itself releases. The defective update is delivered by the same channel to every host the channel reaches in the window of release, and it stops them. It is immediate and unselective: every host that accepts it is affected the same way at the same time, and its blast radius is the population online and

reachable while the release stood. Its recovery is bounded by whether the hosts can still be reached by the channel that broke them. On 19 July 2024 the defective content was released at 04:09 UTC and reverted at 05:27 UTC, seventy-eight minutes later; many hosts that received it in that window failed at boot before they could take the correction and had to be repaired by hand. The vendor's guidance was to restart first, and some hosts took the reverted file before the crash recurred (*The Faulty Content Update of 19 July 2024*, DBA-IT-RE-2). The vendor reported about 99 percent of Windows sensors back online by 29 July, ten days on (CrowdStrike, root cause analysis of 6 August 2024).

The two chains share the channel and differ in two things: who acts, and when the consequence comes. A third case combines them. An adversary who holds the channel can deliver a destructive payload instead of a latent one, and the consequence is then an availability consequence produced by intent, simultaneous like Chain A and chosen like Chain I. NotPetya is that case, and it is cited from the Transportation series rather than proposed again (Section 9.2). It is the bridge to the military-action extension (Section 6.3).

3.3 Design Parameters at Class Resolution

Five parameters govern the envelope. Together they are this class's proposal for the blast-radius measure the framework leaves open (framework §9), and Section 10.2 records them for the framework to take up.

The first is reach: the number and kind of systems a single release of the product can change, stated by the tier of the systems reached and by the sectors they serve. Reach is not the installed base alone. It is the installed base the channel can change in one release, which is smaller where customers stage updates and equal to the installed base where the vendor pushes to all.

The second is privilege: the authority with which what the channel delivers executes on the host, stated as kernel, system, administrative or user, and as whether it can change a control or protective function of a consuming facility. Privilege sets the depth of the consequence on each host reached. Kernel privilege sets whether the host can be recovered without hands at the console. Privilege includes the credentials a product holds and what they reach, identity services and cloud tenancies among them, because the SolarWinds capability inherited the platform's credentials as well as its code. A first- or second-tier product's credentials are scoped to its function, and their use outside that function is made detectable (*The Compromise of the SolarWinds Orion Platform* R-2).

The third is propagation time: the time from release to the point at which the release has reached the share of its reach it will reach. On 19 July 2024 it was the seventy-eight minutes the release stood. Propagation time is what staging controls, and it is the parameter the class can most directly shape (Section 4.4).

The fourth is dwell: the time from the insertion of an adversary's capability, or of a defect, to its detection. In the SolarWinds case the vendor's chief executive said on 19 May 2021 that the actor was doing very early reconnaissance in January 2019, a date the National

Counterintelligence and Security Center's summary repeats, and the vendor's investigation found code designed to test the ability to insert into its builds in a release of October 2019, and the malicious code in releases from 20 February 2020; it was disclosed in December 2020 (SolarWinds, New Findings From Our Investigation of SUNBURST, January 2021). In the XZ Utils case the backdoored releases were published in February and March 2024 found on 28 March 2024 and reported publicly on 29 March, before the stable releases of the major distributions had shipped them. Dwell bounds Chain I in the way propagation time bounds Chain A. Dwell is bounded by the watching. A first- or second-tier product with privileged reach declares the destinations it speaks to and the behavior it shows, is permitted no others, and has departures from what it declared acted on by an authority the consuming enterprise names; the dwell a product line states is the dwell that watching would bound (*The Compromise of the SolarWinds Orion Platform* R-3).

The fifth is time to recovery: the time from detection to the restoration of the systems reached, stated separately for the correction of the product and for the recovery of the hosts. The correction can be minutes; the recovery of hosts that cannot be reached by the channel is set by the number of hands available and by what each host needs, as the ten days of July 2024 show. Where recovery depends on the channel that failed, the parameter is unbounded. It is stated twice, for the hosts and for the service they carry, because a consuming operation can take days longer to rebuild than its hosts take to boot, and it is the service's time that a consuming facility's continuity window is measured against. Recovery keys are held beyond the reach of the same failure (*The Faulty Content Update of 19 July 2024* R-2).

This instrument states the parameters and their governing logic. Their quantitative values for a product line await validation against what vendors already hold: their deployment telemetry, their release records, their published root-cause analyses and their customers' recovery records (Section 10.1). They are not asserted here as numbers, and the figures quoted are the events' own.

4. The Update Channel as the Governing Pathway

4.1 Status and Use

Every event in this class passes through the channel by which the product reaches its hosts, and the channel is the one place the class can govern both chains at once. This section states what the class requires of it. It is written as the physical classes write their governing pathway, the receiving function of the hospital or the release path of the dam, because it is the point at which the class's own consequence becomes the consequence of others. Where a requirement of this section or of Section 5 falls on the vendor, it reaches the vendor as an obligation the asset owner-operator places on it by contract, derived from the consuming facility's design basis, on the author's determination of 14 October 2026 on vendor releases and staged releases (*Three Author Decisions: Continuity Classes, Vendor Releases and Staged Releases*), queued for *The Bright Line* §4.3 in the November cycle. The class states what that contract must require: how the build

is governed, how the release is staged, how it is recovered, and what the vendor tells the consumer of each. A product line serving the tiers that carry the design-basis load meets the obligations the contract would impose whether or not a particular contract yet imposes them.

4.2 Origin Is Not Integrity

A signature tells the consuming system who released an artifact. It does not tell it what the artifact will do. Both of the vendor-side events conferred for this class were genuine in every sense a signature tests: the SolarWinds updates were built by the vendor's own build system, and the faulty content of July 2024 came from the vendor's own pipeline. The class therefore requires that the integrity of a release be established by means other than its signature, and that the signature be taken for what it is, evidence of origin that protects the channel against impersonation and against nothing else.

4.3 The Build as a Governed Act

The two adversarial events conferred for this class share one feature, and it is the class's central finding. In neither was the adversary's code where reviewers would look for it. SolarWinds stated that the vulnerability was not present in the source code repository of the Orion products and was introduced as a result of a compromise of the build system (Form 8-K, 14 December 2020). The XZ Utils backdoor was carried in the upstream release tarballs (National Vulnerability Database, CVE-2024-3094): its payload sat in disguised test files committed to the public repository, and the build script that activated it was present only in the tarball, on the account of the oss-security report of 29 March 2024. In both the artifact shipped was not the artifact the public source would produce. In both a comparison of the shipped artifact against an independent build of the reviewed source would have shown the difference, in the XZ Utils case only because the activating script had been kept out of the repository. A maintainer who had committed that script as well would have defeated the check, and the XZ Utils case therefore sits on the edge of the first residual the class declares (Section 6.8).

The class therefore requires that what ships be shown to be what was reviewed. For a first- or second-tier product the build is to be reproducible, so that an independent party building the reviewed source obtains the same artifact, and the release is to be checked against such an independent build before it reaches the channel. The build system is a system an adversary will reach and is to be protected as one, and a change to what it produces is to be a governed act in the sense of *Local Sovereignty* §3.2: deliberate, authenticated, recorded, and not able to be performed or completed by any automated element alone. Where an upstream component is taken in, it is taken from its reviewed source, or its release is checked against that source, and not taken on the strength of its tarball. The practices to do this exist in the secure development framework the federal government adopted and in the open supply-chain levels the industry maintains (Section 8.4); what the class adds is that, for the tiers that carry the design-basis load, they are design-basis requirements and not maturity goals. Two qualifications from the instrument on XZ Utils sharpen the check. The payload was in the repository, as binary test files

no reviewer could read, so reviewed source has to mean readable source; and release tarballs of projects built this way routinely carry generated files, build macros among them, that the repository does not, so a comparison against them shows differences as a matter of course and a difference that matters is lost among them (*The Backdoor in the XZ Utils Compression Library* F-1). Unreadable content in source is therefore accounted for, its origin recorded and its reach into the build excluded, and releases are generated reproducibly from the repository, or built downstream from the repository rather than from the tarball (*The Backdoor in the XZ Utils Compression Library* R-1). The independent build is independent in fact: the comparison artifact is built from the repository on infrastructure that shares no credentials, administration or network path with the production build, and the comparison is recorded as part of the governed act of release (*The Compromise of the SolarWinds Orion Platform* R-1).

4.4 Staging and the Rate of Propagation

Propagation time is the parameter the class controls most directly. A release that goes to every host at once has a propagation time of its release window and a blast radius of its whole reach. A release staged in rings, first to a small population, held there long enough for a defect to show, then widened, has a blast radius at any moment of the ring it has reached. The vendor's preliminary review of July 2024 committed to a staggered deployment strategy for its rapid content, beginning with a canary deployment, and its witness told the House committee that such content was before the incident sent to all customers in one session and is now released gradually across increasing rings of deployment.

The class requires that every channel by which a first- or second-tier product changes the behavior of its hosts be staged, whether what it carries is called code, content or configuration, and that the first ring be sized and held so that a defect that stops its hosts is seen before the second ring is reached. The distinction between code and content is the vendor's and not the host's: the content of July 2024 was, in the vendor's words, not code or a kernel driver, and it stopped 8.5 million hosts from the kernel. What matters to the design basis is whether a change alters what the host does, and any change that does is staged. The first ring is drawn so that facilities whose service cannot wait, answering points, hospitals and control rooms, are not in it unless they choose to be, and any consuming facility may place itself in a later ring; on 19 July 2024 answering points were among the hosts the single session reached. The vendor states the loss its first ring would carry (*The Faulty Content Update of 19 July 2024* R-1, noted to the consuming classes at Section 10.9).

Staging bounds Chain A. It does little against Chain I, because an adversary's capability is built to pass the first ring unseen, and a latent capability that does nothing in the canary does nothing in the fleet until it is used. Against Chain I the class relies on Section 4.3.

4.5 The Consumer's Authority Over What Enters

A consuming enterprise that stages its updates has authority over what enters its systems only for the channels its staging governs. In July 2024 a customer could hold the vendor's agent at a

version behind the current one, so that new code reached it only after others had run it. The content that failed was not delivered through that mechanism: it was configuration on a channel of its own, and the policy that held back the code did not hold back the content (CrowdStrike, remediation and guidance hub; preliminary post-incident review). The customer's control was real for one channel and absent for the other, and the other was the one that failed.

The class requires that a consuming enterprise's authority over what enters its systems reach every channel by which a product can change what those systems do, and that the vendor give it the means to exercise that authority on each. It requires more at the boundary where the class's logical consequence becomes a physical one. A product that runs on, or can change, a control or protective function of a consuming facility, whether the engineering workstation of a safety system, the firmware of a protective relay or the controller of a plant, is not to be able to alter that function's setpoints, logic or bypass state except through a governed act (*Local Sovereignty* §3.2). An update channel that reaches such a function without one defeats the function's local sovereignty, however local its hardware. A vendor's automatic channel is a reachable authority, and the class's design basis requires that no protective function of a consuming facility be within its reach except through a governed act at the facility.

Whether the consuming facility meets that requirement is the consuming class's to state of its own functions, as the production class states it of its safety controllers (*Chemical Production and Synthesis* §7.2). What this class states is the requirement on what it delivers: that a product line serving such functions offer no path by which its channel can change them without the facility's governed act, and that it say plainly which of its channels can.

4.6 Recovery That Does Not Depend on the Channel

Many of the hosts that failed in July 2024 could not be corrected by the channel that broke them, because they failed before the channel could reach them. Each had to be started in a recovery mode and the faulty file removed by hand, and where the disk was encrypted, the recovery key had to be found first. A fault that disables the means of its own correction converts a seventy-eight-minute release into a recovery measured in days and in the number of people with hands on keyboards.

The class requires that, for a first-tier product with kernel or boot-path privilege, the vendor and the consuming enterprise together hold a recovery path that does not depend on the product's channel or on the host's ability to boot normally, and that the time to recover the enterprise's most critical hosts by that path be stated and exercised. Where the recovery path depends on keys held in a system the same failure can take, the path is not independent. Where the consuming facility is one whose service cannot wait, a hospital, an answering point or a control room, the recovery of its hosts is part of that facility's own continuity, and the time stated here is the figure its continuity window is measured against. The surest recovery path is the one the product does not need: a first-tier product holds its privilege at the lowest level its function needs and states why each function it runs in the kernel needs to be there, and a recovery path the platform offers

is credited only where it is independent of the product and of the public networks (*The Faulty Content Update of 19 July 2024* R-3).

5. Design Basis Events: The Envelope

The events below are the class's design-basis envelope. Each is stated with the event postulated, the consequence threshold, the function the class requires and the duration for which it is required, in the form the physical classes use. The first four are non-adversarial; the last three are adversarial and stand here because the integrity chain is the class's governing chain in every domain, not only under military action. Inclusion is by consequence severity, not likelihood (framework §4.1).

5.1 Defective Update Delivered Through the Trusted Channel

Postulated event a release of code, content or configuration from the vendor's own pipeline that contains a defect causing the hosts that accept it to fail, delivered by the product's channel to every host reachable in the window of release.

Consequence threshold loss of the hosts reached, at the level of the product's privilege, across the release's reach (Chain A); for a first-tier product with kernel or boot-path privilege, hosts that cannot be corrected by the channel.

Required function every channel that changes what a host does is staged (Section 4.4); the release is tested in the configuration in which it will run, inputs and template together, and not only as a type; the consuming enterprise's authority reaches every channel (Section 4.5); and a recovery path independent of the channel is held and exercised (Section 4.6).

Design-basis duration from release until every host reached is recovered; for the staging requirement, the bake time of the first ring.

The case is the faulty content update of 19 July 2024, conferred by the framework as DBA-IT-RE-2 (framework §9.1), whose instrument reads it as non-adversarial. Only Windows hosts running sensor 7.11 or later and online in the window were affected, and Mac and Linux hosts were not; the template type arrived with sensor 7.11 in February 2024. The vendor's root-cause analysis found three things together: a template type defined twenty-one input fields where the sensor's code supplied twenty; the content validator assessed the new content on the expectation of twenty-one; and the interpreter on the host had no check on the bounds of the array it read, so that reading the twenty-first field crashed it. The template type had passed a stress test in March 2024 (CrowdStrike, preliminary post-incident review), and its test content used a wildcard in the twenty-first field, so the fault was never exercised before the release that used it (CrowdStrike, root cause analysis of 6 August 2024). The design-basis reading is that a deterministic component tested as a type was run in a configuration that had never been tested, and that nothing between the vendor's validator and eight and a half million kernels could stop it. Section 7.2 states what that means for the determinism carve-out.

5.2 Latent Defect in a Widely Shared Component

Postulated event discovery, in deployment, of an exploitable defect in a component carried by a large share of first- and second-tier products, so that every system carrying it is exposed at once and the exposure is known to adversaries as soon as to defenders.

Consequence threshold exposure of the population carrying the component, at the privilege of the products that carry it, for the time it takes to find and correct every instance (Chain I opened to any adversary).

Required function every first- and second-tier product states what components it carries, so that a consuming enterprise can tell within hours whether it is exposed; the vendor's correction reaches the channel at the rate of Section 4.4 with the exposure, not the defect, setting the urgency; and the consuming enterprise holds a means of containing an exposed product that does not wait on its correction. The exposed population is known from a record and not from a search: the vendor records which release each customer took and when, and the consuming enterprise records which release each of its systems runs (*The Compromise of the SolarWinds Orion Platform* R-4).

Design-basis duration from disclosure until the last exposed instance in a consuming enterprise's first- and second-tier systems is corrected or contained.

The case is the defect in the Log4j logging library disclosed in December 2021, for which the Cybersecurity and Infrastructure Security Agency issued an emergency directive to federal agencies within days. The component was carried inside products whose own vendors did not at first know it was there. The design-basis reading is that reach is set by what links to a component, and that a consuming enterprise that cannot list what its products contain cannot bound its own exposure.

5.3 Loss of the Vendor, the Maintainer or the Right to Update

Postulated event a product's updates stop because its vendor fails, its maintainers leave, or a government prohibits its vendor from supplying updates, so that the installed base is left unmaintained or must be replaced.

Consequence threshold a first- or second-tier product left without correction for defects found after its last update, or a forced replacement across the installed base on a timetable set by the event rather than by the consuming enterprise.

Required function for each first- and second-tier product, the consuming enterprise states how it would operate the product unmaintained and how long it would take to replace it; and no protective or control function of a consuming facility depends on a product whose loss of updates would stop it.

Design-basis duration the time to replace the product across the consuming enterprise's first- and second-tier systems.

The case is the prohibition the Department of Commerce issued on 20 June 2024 against the security products of Kaspersky Lab, under which the company could provide antivirus signature and code updates to United States customers only until 29 September 2024. The determination gave as one ground that the company's privileged and persistent access to its customers' devices could be used to install malicious software or to withhold critical malware signature updates. The design-basis reading is that a government can end a product's updates on a date, and that the same privilege that makes a security product useful makes its vendor's jurisdiction part of the consumer's design basis (Section 6.1).

5.4 Loss of the Signing or Distribution Infrastructure

Postulated event a vendor's signing key is lost, revoked or expires, or its distribution servers, mirrors or registry become unavailable, so that legitimate updates cannot be released or cannot be verified by the hosts that must accept them.

Consequence threshold a first-tier product unable to deliver a correction during an event that requires one, or hosts that refuse legitimate updates or stop running signed code (Chain A, by the absence of a release rather than by a defective one).

Required function the vendor holds its signing keys so that no single failure or compromise ends its ability to sign, and holds a means of re-establishing trust in its channel that does not depend on the channel; the vendor's distribution survives the loss of any one provider of hosting or delivery.

Design-basis duration the time to restore a trusted release path.

5.5 Insertion Through the Build or Distribution

Postulated event an adversary gains access to the vendor's build system, signing or distribution and causes the vendor's genuine channel to deliver, under the vendor's signature, an artifact carrying the adversary's capability that is not present in the reviewed source.

Consequence threshold adversary capability installed across the release's reach and executing with the product's privilege, latent until used and used selectively (Chain I).

Required function the shipped artifact is shown to be what the reviewed source produces, by an independent build (Section 4.3); a change to what the build system produces is a governed act; and the consuming enterprise limits what the product can reach to what its purpose requires, so that the capability inherits as little reach as the product needs. And the product's declared destinations and behavior are watched, so that a capability that speaks where the product does not is seen in the time Section 3.3 states (*The Compromise of the SolarWinds Orion Platform* R-3).

Design-basis duration the dwell of the capability, from insertion to detection, and thereafter until every installation is removed and every system it reached is cleared.

The case is the compromise of the SolarWinds Orion platform, conferred by the framework as DBA-IT-RE-1 (framework §9.1). The vendor's chief executive dated very early reconnaissance by the actor to January 2019, a date the National Counterintelligence and Security Center's summary repeats. The vendor found the earliest suspicious activity on its internal systems in September 2019, a release of October 2019 that appeared designed to test the ability to insert code into its builds, and the malicious code inserted into releases from 20 February 2020; the code was removed from its environment in June 2020 (SolarWinds, New Findings From Our Investigation of SUNBURST, January 2021), and the affected updates were those released between March and June 2020 (Form 8-K, 14 December 2020), a security firm's report finding the trojanized updates signed from March to May 2020. On 13 December 2020 the Cybersecurity and Infrastructure Security Agency directed federal agencies to disconnect or power down the affected products (Emergency Directive 21-01). The United States government attributed the campaign to the Russian Foreign Intelligence Service on 15 April 2021; the attribution is stated as that government's, and this instrument makes no finding of attribution. The design-basis reading is that a platform trusted to manage networks delivered an adversary into as many as eighteen thousand of them, and that follow-on compromises were disclosed by nine federal agencies and about a hundred companies, on the account of the National Counterintelligence and Security Center. That reach bounds what the campaign did through this product; some victims' cloud compromises predated the backdoor, their initial vector unknown, on the testimony of the acting director of the Cybersecurity and Infrastructure Security Agency of 18 March 2021.

A neighboring form, noted and not proposed, is the ransomware pushed through the Kaseya VSA remote-management product in July 2021. It was not an insertion into the build: the attackers exploited previously unknown flaws in customers' on-premises VSA servers and then used the product's own functions to push their payload, and the vendor reported no evidence that its code base had been modified. It reached fewer than 1,500 downstream businesses through fewer than sixty of the vendor's direct customers, on the vendor's account, many of them managed service providers. It shows the channel abused rather than corrupted, and the integrity chain run at one remove: the vendor's customers were themselves the channel to their own customers, so that the reach of one product was multiplied by the reach of each provider that ran it.

5.6 Compromise of an Upstream Dependency or Its Maintainers

Postulated event an adversary becomes, or replaces, a maintainer of a component that first- or second-tier products carry, and releases a version carrying the adversary's capability, which the products and distributions downstream take in as a routine update.

Consequence threshold adversary capability carried into every product and distribution that takes the version, executing with the privilege of the service that links it (Chain I), across a reach set by what links to the component rather than by the component's project.

Required function a component is taken from its reviewed source or checked against it, not taken on the strength of its release package (Section 4.3); a first-tier product states which of its

components are maintained by too few people for their integrity to be assured, and carries those at a version reviewed by its own staff; and the distribution that packages a component checks its release against its source before carrying it. The receiver sets the pace of intake: a first-tier component enters stable releases only after a holding period in development use, an upstream request to hurry it in is recorded and scrutinized, and the hold is credited only as time for detection, not as a defense in itself (*The Backdoor in the XZ Utils Compression Library* R-2). A change in who can release a thinly maintained component is a change to the product: who holds release authority is recorded, and a change of those people is reviewed, as a governed act, before their next release is taken (*The Backdoor in the XZ Utils Compression Library* R-3).

Design-basis duration the dwell of the capability, and thereafter until every product and distribution that took the version has removed it.

The case is the backdoor placed in the XZ Utils compression library, conferred by the framework as DBA-IT-RE-3 (framework §9.1). The backdoored versions, 5.6.0 and 5.6.1, were published in February and March 2024. The malicious code was in the upstream release tarballs and was assembled, through a series of obfuscations, from files in the release during the library's build, producing a modified library that any software linked against it would carry (National Vulnerability Database, CVE-2024-3094, rated 10.0). The contributor who placed it had become a maintainer of the project over a period of years. An engineer at Microsoft found it while investigating why logins to a development system were slow and using unusual processing, on 28 March 2024, notified the distributions privately, and reported it publicly on 29 March. By then it was in the development and testing branches of several distributions, among them the Fedora 40 beta and Debian's testing and unstable branches, and had reached users of rolling-release distributions such as openSUSE Tumbleweed, Kali and Arch; the stable releases of the major enterprise and long-term distributions had not taken it, and the Cybersecurity and Infrastructure Security Agency advised users to move to an uncompromised version the same day. The design-basis reading is that a component maintained by very few people sat beneath a privileged service on a large share of the world's servers, that the defense was a release cycle that happened to be slower than the adversary's schedule, and that the detection was a chance observation. The class does not take credit for either.

5.7 Counterfeit or Tampered Hardware and Devices

Postulated event hardware or a device enters a consuming facility counterfeit, altered in transit, or carrying firmware other than its manufacturer's, so that a component trusted as genuine behaves as an adversary or a defect directs.

Consequence threshold at the depth of the sub-index only; to be developed (Section 10.3).

Required function at the depth of the sub-index: the firmware is governed by Sections 5.1 to 5.6; the provenance of first- and second-tier hardware is traceable to its manufacturer; and the physical device is held for a later revision.

Design-basis duration to be developed.

Table 5-1. Facility-class design-basis event categories.

Event category	Postulated event (class-specific)	Consequence chain	Framework source
Defective update through the trusted channel	Vendor's own release stops the hosts that accept it; hosts unrecoverable by the channel	Chain A	§4.1 candidate (supply-chain compromise); developed here
Latent defect in a widely shared component	Exploitable defect disclosed in a component most products carry	Chain I (opened to any adversary)	developed here
Loss of the vendor, maintainer or right to update	Updates end by failure, abandonment or prohibition	Chain A (by absence)	developed here; §6.1
Loss of signing or distribution infrastructure	Key lost, revoked or expired; distribution unavailable	Chain A (by absence)	developed here; §3.3 (trust anchors)
Insertion through the build or distribution	Adversary code shipped under the vendor's signature, absent from source	Chain I	§4.1 candidate (inserted into a trusted update)
Compromise of an upstream dependency	Adversary becomes or replaces a maintainer and releases a backdoored version	Chain I	§4.1 candidate; developed here
Counterfeit or tampered hardware and devices	Device counterfeit, altered in transit or carrying foreign firmware	Chain I	§3.4 (hardware and device supply); sub-index depth

6. Military-Action Extension: DBA-MA for the Supply Chain

6.1 MA Extension Applicability Determination

Every first- and second-tier product line in the class performs the applicability determination the series makes mandatory, under the two-factor test of geolocation and strategic value, with strategic value driven by blast radius (framework §7). For a logical class the first factor needs restating, because the product has no site of its own. The geolocation that matters is threefold: where the product is built, signed and distributed, which sets where an adversary must reach to hold its channel; where its installed base is, which sets which theater its consequence reaches; and under whose law its vendor operates, which sets whether a government can compel the channel's use or its withdrawal. The Department of Commerce's determination of June 2024 rested in part on the last of these (Section 5.3). Strategic value is the product's tier and the

sectors its reach includes. The framework's presumption toward applicability is strong for the most widely deployed software (framework §7), and this instrument reads that presumption as reaching every first-tier product and every second-tier product that can change a control or protective function of a consuming facility. The determination is recorded, it is time-varying and re-triggered on the indicators the DBA-MA parent framework defines, and any scope reduction carries a recorded basis (Section 6.9).

6.2 The Four Characteristics Applied to the Class

Applied to the supply chain, the four military-action characteristics take specific form, in the canonical statement of *Design Basis Layers* §4.4. Intentionality: the adversary chooses the product for its reach into the target, and may care nothing for the vendor or for most of the product's customers. Persistence: the adversary holds the channel, or a position in the product's path, across the time needed to reach the moment it wants, which in the SolarWinds case ran from reconnaissance the vendor's chief executive dated to January 2019, and from the test release of October 2019, to disclosure in December 2020 and in the XZ Utils case from the contributor's cultivation of the project over years to the release of 2024. Precision: the capability is used selectively, on the systems and at the time the adversary chooses; the SolarWinds capability was installed in up to eighteen thousand networks, and follow-on compromises were disclosed by nine federal agencies and about a hundred companies, and the XZ Utils backdoor was built so that only the holder of a particular key could use it, on the analyses that followed the oss-security report. Denial: the adversary uses the channel to deliver destruction rather than access, or times its use to a moment when the consuming facilities cannot recover, so that the hosts are lost when they are needed and the hands to restore them are needed elsewhere.

Under the author's rulings of 10 and 13 October 2026, the military-action tier is not met without denial, and all four characteristics must be established on the record; a characteristic not determined counts as not present (*Design Basis Layers* §6.4). On the author's ruling of 14 October 2026 a state's surveillance and covert theft operations are military actions (*Design Basis Layers* §6.4), so an insertion by a state that shows intentionality, persistence and precision and is used for espionage is recorded as military action that realizes the scenario it realizes, without the tier, as DBA-IT-RE-1 records SolarWinds. The tier of each reference event is its instrument's to determine, not this class's.

6.3 The Pre-Positioned Capability Activated as the Governing MA Scenario

The governing military-action scenario is a capability pre-positioned in a first-tier product through the integrity chain and activated at the onset of a conflict as a destructive update, timed to an attack on the grid or the communications of the same region. It joins the two chains. The capability was delivered quietly, months before, to every system that installed the product; on activation it stops or destroys all of them at once, as a defective update would, but on the adversary's day; and the recovery of Section 4.6, which in peacetime is limited by the number of hands available, is now limited by hands already engaged in restoring power and

communications. No credit is taken for the vendor's correction, which the adversary may have anticipated or may deny by striking the channel itself, or for recovery that depends on the networks the same campaign takes.

The extension therefore requires three things. It requires that the consuming facilities' protective and control functions be beyond the product's reach except through a governed act (Section 4.5), so that the activation stops the logical layer without reaching the physical one. It requires that the most critical hosts of each consuming facility be recoverable by a path that needs neither the channel nor the public networks, by the facility's own staff, in a time the facility's continuity window can bear (Section 4.6). And it requires that the vendor's own release path be one an adversary inside it cannot use alone, by the build discipline of Section 4.3.

The empirical case is NotPetya in June 2017, delivered through the update system of a Ukrainian accounting program into the networks that ran it and spreading from them; the governments of the United Kingdom and the United States attributed it to the Russian military and described its primary targets as Ukrainian, and it reached companies on several continents through their offices in Ukraine (*The NotPetya Attack on Maersk's Terminals* §2.3, F-2, F-8). The instrument records that a backdoor was in the supplier's updates from April 2017, on Cisco Talos's account, which is the pre-positioning this scenario postulates. It is cited, not proposed again. The federal government's advisory of February 2024 on state actors pre-positioning in critical infrastructure networks for disruption in a future crisis (Cybersecurity and Infrastructure Security Agency and partners, AA24-038A) describes the intent this scenario postulates, by another route than the supply chain. This scenario is the one the anticipated use case of this class drives (Section 9.2).

6.4 The Channel Taken or Withdrawn in Conflict

The second military-action scenario is the loss of the channel itself. An adversary may strike the vendor's build and distribution, or the vendor may be in the theater: the accounting software through which NotPetya travelled was written and distributed in the country under attack. Or the channel may be withdrawn: a vendor subject to one government may be required to stop updating the systems of another, or may choose to. In either case the installed base is left unmaintained at the time it is most likely to need correction. The class requires a product line with a positive applicability determination to state where its build, signing and distribution are, how its release path continues if any one of them is lost, and what its customers would do if its updates stopped, as Section 5.3 requires in peacetime; and it requires the consuming facility of a second-tier product that controls or protects a physical process to be able to operate the process on that product's last good configuration for the duration of the conflict.

6.5 Hardware and Device Interdiction

The hardware and device supply carries a military-action scenario of its own: devices interdicted between manufacturer and user and altered to carry a payload, physical or logical, that the adversary can trigger. It is stated at the depth of the sub-index and held for the later revision of Section 10.3, which is to read the hardware cases against this class and against the critical-

manufacturing series. The detonation of pagers and radios in Lebanon in September 2024 is the case most often cited for the physical form, and it is recorded here as a candidate for that revision and not proposed as a reference event.

6.6 Cross-Sector Interaction

Under military action the class is tied to every sector in one direction and to four in the other. Going out, a first-tier product reaches the answering points, hospitals, banks, airlines, utilities and government offices that run it, and each meets the activation as the loss of its logical layer, with the consequence its own class states: the answering point's dispatch, the hospital's record and orders, the control room's displays. Coming in, the class depends on electric power and the data centers for its builds and its distribution, on communications for its channel, on the certificate authorities of the connectivity class for the trust its signatures carry, and on critical manufacturing for the hardware on which every product runs (framework §6.1). An adversary need not attack a facility to defeat it; an attack on the product the facility trusts, timed to an attack on what the facility consumes, is enough.

6.7 The Adequacy Result: Bounded Propagation Condition

The test of whether the class's protection is adequate is whether a product line and its consuming facilities can reach what this document calls the Bounded Propagation Condition. When the product's channel delivers a defective or hostile release, the release reaches no more than the first ring before it is seen; what it reaches cannot change a control or protective function of any consuming facility except through that facility's governed act; and every host it reaches can be restored by a path that needs neither the channel nor the public networks, within a time stated for each consuming facility's most critical hosts. The condition bounds the blast radius in reach, by staging; in depth, by the independence of the consuming facilities' protective functions; and in time, by independent recovery.

The protective functions on which this condition finally rests are not the class's own. They are the protective functions of the consuming facilities, which are to be locally sovereign as FD-LS defines the property where they are protective functions in that sense (*Local Sovereignty* §3.2): a measurement against a setpoint, colocated with what it protects, which no reachable system can stop, delay or trigger, and whose setpoints, logic and bypass state change only through a governed act. A vendor's update channel is a reachable system. This class states the requirement as a requirement on what it delivers, and each consuming class states it of its own functions. Local Sovereignty governs and is not restated here. It describes what protects the consuming facility when the products it runs have failed or turned; it does not decide whether any action needs human authorization (*Local Sovereignty* §4).

The end state may also be reached by disconnection. Taking a product off the network, or powering down the systems it runs on, ends its reach at the cost of its function, as the emergency directive of December 2020 required of the federal agencies running the affected platform. Like the evacuation of a hospital or the drawdown of a dam, it is a protective action with an authority

named in advance, a trigger stated in advance, and a statement of what the facility loses while the product is gone.

6.8 Adequacy and the Declared Residual

In the non-adversarial domain, the Bounded Propagation Condition is achievable with the Section 4 and Section 5 requirements: staged channels with the consumer's authority over each, releases tested in the configuration in which they run, and an independent recovery path held and exercised. Against a deliberate adversary, adequacy has limits.

Three residuals are declared, in the output form of *Defense in Depth* §3.5. First, a capability placed in the reviewed source by a trusted maintainer, and therefore present in every independent build, passes the check of Section 4.3; beyond the design act the residual is adversary capability in the installed base, and the response is the limitation of what the product can reach and the independence of the consuming facilities' protective functions, not the prevention of the insertion. Second, the first ring of a staged release is real hosts in real enterprises; staging bounds the blast radius and does not remove it, and the residual is the first ring's loss, which the vendor's choice of first ring must state. Third, a vendor compelled by its government to use or withdraw its channel acts with the channel's full authority, and no discipline inside the vendor holds against it; the residual is the product's whole reach, and the response is the applicability determination of Section 6.1 and the consuming enterprise's ability to replace the product (Section 5.3). These residuals bound the class adequacy claim.

6.9 Consequence-Basis Conditionality Check

Any relaxation of a conservative class parameter, such as a channel exempted from staging because what it carries is called content, a first ring sized for the vendor's convenience rather than for the bake time a defect needs, a recovery path credited that depends on the channel or on keys held where the same failure reaches, or an upstream component taken on its release package because its project is small, must be justified against the class baseline, and the larger the relaxation the stronger the analytical basis required. A relaxation without a recorded basis is not consistent with the class design basis. The framework does not yet state this principle for the sector, and Sections 9.1 and 10.9 record it for the framework to adopt, as the transportation framework has and as the healthcare class asked of its own framework.

7. The Dual Design Basis, Stated in Full

7.1 The Two Design Bases

The class carries two design bases at once: a product-integrity design basis, defined in this instrument, and an AI-governance design basis, governed by the Foundational Definitions and the Command Broker family and named and referenced here (framework §5). In a physical class the first is the physical design basis of the plant. A logical class has no plant, and its first design basis is the set of deterministic controls on the product's path from source to host together with

the requirement it places on the consuming facility's protective functions. The two design bases interact at the point developed in Section 7.4.

7.2 The Product-Integrity Design Basis: The Determinism Posture

The product-integrity design basis is the set of controls the class relies on to keep what reaches the host what was intended: the independent build and its comparison with the release (Section 4.3), the staging of every channel (Section 4.4), the consumer's authority over each channel (Section 4.5), and the recovery path that does not depend on the channel (Section 4.6). Most of these are deterministic functions, and some are the kind of function the determinism carve-out was written to protect.

Whether a function is deterministic decides whether the authorization rules apply to it at all. It does not decide where its actions fall on the Bright Line. Components that behave predictably within a stated range of inputs, and that have been formally tested in the deployed configuration, are outside the Command Broker obligation for the actions they take while their inputs stay in that range (*Layer Decomposition and Determinism* §4.3). They are not below the line; they are outside the obligation only because, and only while, their behavior is bounded by that showing (*The Bright Line* §4.5).

The class's events show how that showing fails. The content interpreter of July 2024 was deterministic. Its template type had been tested. The configuration it ran on the morning of the release, the new template instance with a non-wildcard value in its twenty-first field, had not been, and the release changed that configuration on every host at once. The author determined on 8 October 2026 that the carve-out does not cover a carved-out function while it runs a configuration other than the one that was formally tested, including a configuration altered outside a governed act (Determination of 8 October 2026, A Protective Function Running Logic Other Than What Was Tested Is Not Covered by the Carve-Out; for *Layer Decomposition and Determinism* §4.3 at its next revision). The SolarWinds updates are the second form of the same condition: the platform as delivered was a configuration altered outside a governed act. A component whose configuration a channel changes many times a day cannot claim the carve-out on the strength of a test of its type. It holds the carve-out only for the configuration that was tested, and a product line that wishes to rely on it must test each configuration its channel will deliver, in the form in which it will run, before the channel delivers it. A component that has not been shown to meet the test is covered by the rules, and each of its actions is judged on its consequence. Where that component is a protective function of a consuming facility, losing the carve-out does not put a Command Broker on it; it means the exemption no longer vouches for the function, which must then be treated as a protective function that may have failed (the same determination).

7.3 The AI-Governance Design Basis: Named and Referenced

Machine-learning and generative-AI systems now sit throughout the supply chain: assistants that write code which ships, systems that generate detection content for security products, agents that

propose and merge dependency updates, systems that review changes before they reach the build, and agents that push remediation to fleets of hosts. The framework names the sector as the one where autonomous AI most directly becomes the threat it defends against (framework §5). These systems are governed by the Bright Line and Command Broker architecture (framework §5) and by the Command Broker family, whose parent is CB-Framework_Cross_Sector (*Command Broker Framework*); CB-REG (*Command Broker Family Register*) is authoritative for the family's version state. The framework makes the coordination class, OpsCo, the locus at which the sector's placement and obligation are implemented (framework §3.5). Placement is by the consequence of each action, not by the component (*The Bright Line* §4.2, §4.3).

Applying that placement to this class raised a question the Foundational Definitions did not answer. This instrument stated its reading at v0.1 and referred the question to Project 0, and the author decided it on 14 October 2026. *The Bright Line* §4.3 defines one action, for the industrial Line, as one change a component's output would make to a process parameter, to a protective function's settings, trip logic, bypass state or actuation criteria, or to the pre-authorized envelope or the gatekeeper. A vendor's release of an update to millions of hosts is not obviously such a change, and the decision has two parts, on the consuming side and on the vendor's.

On the consuming side, the decision confirms the class's reading that a release that would change the logic, setpoints or bypass state of a control or protective function in a consuming facility is, at that facility, a change of the kind *The Bright Line* §4.3 names. Where an in-scope component proposes or produces it, it is an action placed on its consequence at the facility; and whether or not any in-scope component is involved, the change must come through a governed act for the function to remain locally sovereign (*Local Sovereignty* §3.2). The requirement of Section 4.5 is the second of these, and it stands whatever the answer to the first.

On the vendor's side, *The Bright Line* §4.3 is not extended to a vendor's release to a population of hosts, on the author's determination of 14 October 2026 on vendor releases and staged releases (*Three Author Decisions: Continuity Classes, Vendor Releases and Staged Releases*), queued for *The Bright Line* §4.3 in the November cycle. The controls on the vendor's side are obligations the owner-operator places on the vendor by contract, derived from the facility's design basis, and Section 4.1 states how the class carries them: the update-channel requirements of Section 4 are what the contract must require. The release is not placed as a Bright Line action of the vendor. A generative system that writes content, an agent that approves its promotion, or a system that decides when a ring is widened is still within the class's requirements, through the governed acts of Sections 4.3 and 7.2 that the contract binds the vendor to. On staging the same determination adopts the reading this class proposed: each widening of a staged release is its own action, and the first release is placed on its whole planned reach, so that staging cannot split a release into steps that each sit below the line. That holds within an owner-operator's own fleet, where the release is the owner-operator's act, and in the staging terms a contract places on a vendor. At the facility nothing changes: taking in a release that would change a control or

protective function is a §4.3 action placed on its consequence, and it comes in only through a governed act (*Local Sovereignty* §3.2).

Two further points apply whatever the answers. Any output on which a release decision is grounded, the result of a validator, a test run or a canary's telemetry, carries the indication-integrity obligation of *Indication Integrity*: a canary reported healthy by the same channel that may be failing is not a basis for widening the release until its health can be corroborated by an independent path. And the build system's output, the signing and the staging of a channel change only through a governed act, by whatever component a change is proposed, because those are the controls on which the product-integrity design basis rests. That is a requirement of the product-integrity design basis in the sense of *Local Sovereignty* §3.2, not a placement: where an in-scope component proposes such a change, the change is also an action placed on its own consequence (*The Bright Line* §4.3).

7.4 The Interaction Point: The Release Meets the Protective Function

The two design bases meet where a release reaches a consuming facility's control or protective function. The product-integrity design basis guarantees, so far as it can, that what arrives is what was intended. The AI-governance design basis guarantees that no proposing system can release, widen or alter what arrives without a qualified human authorization the architecture can enforce, where the action's consequence requires it. And the consuming facility's protective functions must remain beyond the reach of either except through a governed act, so that when both have failed, the plant is still protected by physics. The products fail, the channel turns, and the facility's protection holds because it never depended on them.

7.5 Scope Boundary

This instrument states the product-integrity design basis in full and names the AI-governance design basis without restating it. The Foundational Definitions own the Bright Line, its scope and its obligation; the Command Broker family owns the Command Broker; the coordination class owns the implementation of placement in the sector; and the consuming classes own the placement of actions inside their facilities. This instrument applies them to the supply chain and, where their application is unsettled, refers the question.

8. Cross-Sector Dependencies and Regulatory Context

8.1 What Other Sectors Depend On: the Trusted Product

Every class in the corpus runs software, and every one of them consumes this class. The class's outward dependency is the broadest of any in the corpus, because a first-tier product reaches all of them at once (framework §6.2). The events of this class reached, on the record: emergency services, whose 911 centers in Phoenix and Portland answered calls by hand in July 2024, on the Congressional Research Service's account (IF12717); healthcare, financial services and federal agencies, whose hospitals delayed and cancelled procedures and whose banks lost transaction

processing that day, on the statements of the subcommittee's chairman at the House hearing; transportation, whose flights were grounded and whose container terminals closed in 2017; and government, whose agencies were the first named victims of the SolarWinds campaign. The consequence in each is the consuming class's to state, and the earlier classes of this project already state several: the loss of the hospital's information systems (*Care Delivery* §5.4), the loss of a shared software platform at the transportation node (*Nodes and Terminals* §5.5), and the cyber attack on dispatch in the answering-point class (*Public-Safety Answering and Dispatch*, DBA-ESS-AD-FC1).

8.2 What the Class Depends On

The class depends on electric power and the data centers on which its builds run and from which its updates are served; on communications for its channel; on the certificate authorities of the connectivity class for the trust its signatures carry; and on critical manufacturing for the hardware on which every product runs (framework §6.1). It depends also on people the corpus has not counted elsewhere: the maintainers of open-source components, many of them unpaid and many of them alone, on whose integrity and endurance products of the first tier rest. The XZ Utils case shows that an adversary can count them too.

8.3 The Boundaries with the Partner Classes and Series

Five boundaries are drawn. With *Infrastructure and Compute* (DBA-IT-IC-FC1): where a vendor runs the console or the cloud service from which its product is managed and its content delivered, the service is that class's and what it delivers into the host is this class's; a security product that is both an agent on the host and a service in the vendor's cloud is in both, and Section 10.6 records the case. With *Connectivity and Protocols* (DBA-IT-CP-FC2): the certificate authority that issues a code-signing certificate is that class's, and the vendor's custody and use of its key are this class's. With the coordination class, OpsCo: the managed service providers and security operations centers that run products on their customers' behalf are its members, and the products they run are this class's; the Kaseya case shows the two joined (Section 5.5). With the critical-manufacturing series: the fabrication of semiconductors is that series', and the integrity of firmware and devices built on them is this class's, at the depth of Section 2.2. With every consuming class: the consuming class governs what the product does in its facility and the placement of actions there; this class governs what the product brings in.

8.4 Regulatory and Institutional Context

The class is governed by frameworks and attestations more than by a regulator. Executive Order 14028 of 12 May 2021, Improving the Nation's Cybersecurity, directed the federal government to set standards for the security of the software it buys, and the National Institute of Standards and Technology's Secure Software Development Framework (Special Publication 800-218, February 2022) states the practices; federal suppliers attest to them under the Office of Management and Budget's memorandum M-22-18 and the Cybersecurity and Infrastructure

Security Agency's secure software development attestation form. The same agency's secure-by-design program, and the pledge many vendors signed in 2024, set goals for vendors' own practice. The industry's Supply-chain Levels for Software Artifacts specification sets graded requirements for build integrity and provenance, and the software bill of materials, whose minimum elements the National Telecommunications and Information Administration published in July 2021, is the means by which a consumer can list what a product contains. For industrial control systems, IEC 62443-4-1 sets requirements for a secure product development lifecycle. In the electric sector, the reliability standards on supply-chain risk management and configuration change management carry the consumer's side into a mandatory regime. In Europe, the Cyber Resilience Act, Regulation (EU) 2024/2847, sets essential requirements for products with digital elements, with reporting obligations applying from September 2026 and its main obligations from December 2027.

The regulatory gap is the framework's (framework §2.3). The regimes state practices, attestations and maturity; none states a design basis in terms of the consequence a product's reach can carry. None sets a rate at which a first-tier product may change its hosts, or requires that a channel carrying configuration be staged like one carrying code, or requires that a consuming facility's protective functions be beyond a vendor's channel. The vendor's own commitments after July 2024 supplied two of these for one product. This class states them as design-basis requirements for the tiers that carry the load.

9. Inheritance Map and Anticipated Derivatives

9.1 The Inheritance Map

The series requires every facility-class instrument to state, at first publication, which elements it inherits unchanged, which it inherits with adaptation, and which it defines locally. Facility-class instruments inherit the framework's methodology and architecture and add their function-specific consequence chain, expressed in blast-radius terms (framework §8.1). Table 9-1 is the map for this class. Where the framework is a skeleton, the element is developed here and recorded for the framework to take up.

Table 9-1. Inheritance map for DBA-IT-SW-FC3.

Element	Source in the framework	Inheritance mode
Function-primary placement, SW function class	§1.3, §3.1, §3.4	Inherited unchanged
Logical class type	§3.1; determination of 30 September 2026	Inherited unchanged; first logical class written (Section 1.4)
Foundational-dependency sub-index	§3.1, §8.2	Inherited with adaptation; three tiers stated, hardware and devices at sub-index depth (Section 2.2)

Element	Source in the framework	Inheritance mode
Design-basis principles (consequence severity, not likelihood)	§4.1	Inherited unchanged
Blast-radius consequence measure	§3.6 (proposed); §9 (open)	Developed here as five parameters (Section 3.3); for the framework to take up
Event categories	§4.1 (candidates)	Developed here ahead of the framework; for the framework to take up
Bright Line and Command Broker architecture	§5; FD-BL; Command Broker family	Inherited with adaptation: the class's reading of an action for a release, referred at v0.1 and decided on 14 October 2026; the vendor's controls carried by contract (Sections 4.1, 7.3)
Update channel as the governing pathway	§3.4	Locally defined (Section 4)
Bounded Propagation Condition	§7 (skeleton)	Locally defined (Section 6.7)
Declared residuals and conditionality check	not yet in the framework	Locally defined; recorded for the framework (Sections 6.8, 6.9)
Digital-triad and manufacturing boundaries	§1.2, §6.3	Inherited unchanged (Section 8.3)
Military-action applicability	§7	Inherited with adaptation: geolocation restated for a product without a site (Section 6.1)

9.2 Anticipated Derivatives

Product-line, vendor and consuming-enterprise design bases inherit this class document by reference and add only what is locally specific, each carrying a recorded applicability determination for any scope it reduces. The three reference events this class proposed at v0.1, on the author's decision of 13 October 2026, are conferred by the framework at §9.1: DBA-IT-RE-1, *The Compromise of the SolarWinds Orion Platform*, 2019 to 2020 (Section 5.5); DBA-IT-RE-2, *The Faulty Content Update of 19 July 2024* (Section 5.1); and DBA-IT-RE-3, *The Backdoor in the XZ Utils Compression Library*, 2024 (Section 5.6). Their instruments were written by Project 11 on 14 October 2026, and this revision takes in their two errata and ten recommendations. The NotPetya attack of June 2017 is cited from the Transportation series as *The NotPetya Attack on Maersk's Terminals* (DBA-TRAN-RE-1) and is not proposed again

(Section 6.3). The facility-class use case for the governing military-action scenario of Section 6.3, a pre-positioned capability in a first-tier product activated during an attack on the grid, is DBA-IT-UC-1, issued under this revision in the fifth wave of the plan for closing the seven sectors.

10. Gaps and Open Items

10.1 Quantitative Design Parameters

The five parameters of Section 3.3 are stated without values. Their values for a product line are to be validated against the vendor's deployment telemetry and release records, its published root-cause analyses, and its customers' recovery records; for the first ring of Section 4.4, against the time a defect of each kind takes to show in a host population.

10.2 The Blast-Radius Measure

The framework leaves the blast-radius consequence measure open (framework §9). Section 3.3 proposes it as five parameters, reach, privilege, propagation time, dwell and time to recovery, and not as a single number, because the two chains are bounded by different parameters: Chain A by propagation time and time to recovery, Chain I by dwell and by what the product can reach. Whether the framework adopts the five, reduces them, or combines them into an index is the framework's to decide, and the other two logical classes of the sector should be read against them before it does.

10.3 The Hardware and Device Supply Chain

On the author's decision of 13 October 2026, hardware and devices are stated at the depth of the sub-index. A later revision is to develop them: counterfeit and tampered network equipment and servers, firmware shipped other than the manufacturer's, devices interdicted in transit, and the provenance of the field devices of control systems; with the boundary to the critical-manufacturing series, which holds semiconductor fabrication and whose Precision and Electronics class (DBA-CM-PE-FC3) is anticipated; and with the detonations of September 2024 in Lebanon (Section 6.5) and the reported findings of undocumented communication components in imported power inverters among the cases to be read.

10.4 What One Action Is for a Release

The two questions this class referred at v0.1 are decided, on the author's determination of 14 October 2026 on vendor releases and staged releases (*Three Author Decisions: Continuity Classes, Vendor Releases and Staged Releases*), queued for *The Bright Line* §4.3 in the November cycle. *The Bright Line* §4.3 is not extended to a vendor's release to a population of hosts; the vendor's controls are contractual, and Sections 4.1 and 7.3 state how the class carries them. A staged release is placed on its whole planned reach, and each widening is its own action,

as this class read it. Until the text enters *The Bright Line* §4.3, the determination is cited. The consuming-side requirement of Section 4.5 rests on *Local Sovereignty* §3.2 and is unchanged.

10.5 Local Sovereignty in the Consuming Facility

The Bounded Propagation Condition rests on protective functions that are not this class's (Section 6.7). This instrument states the requirement on what the class delivers. Whether each consuming class states, of its own functions, that no vendor channel can reach them except through a governed act is for each to confirm at its next revision. Several already state the substance: the production class for its safety controllers (*Chemical Production and Synthesis* §7.2) and the care-delivery class for a drug library pushed to a fleet of pumps (*Care Delivery* §7.2). Whether the DBA-MA parent framework's local-sovereignty principle is the same thing FD-LS defines remains the open question the other classes record.

10.6 The Boundary with Infrastructure and Compute

A security product that runs as an agent on the host and is managed and fed from the vendor's cloud is in two classes at once, and the line between them is drawn at Section 8.3 by what each governs: the service is the compute class's, and what the service delivers into the host is this class's. The compute class is not yet written. When it is, the line should be checked from its side, and the case of a channel whose content is generated and released in the cloud should be stated in both.

10.7 Reference Events and the Use Case

The three reference events this class proposed are conferred by the framework at §9.1 as DBA-IT-RE-1 to DBA-IT-RE-3, and their instruments were written by Project 11 on 14 October 2026. DBA-IT-RE-1 records SolarWinds as military action without the military-action tier, three characteristics present and denial not present, with Government Facilities entered as an impacted sector on the author's decision of 14 October 2026; DBA-IT-RE-2 records the content update as non-adversarial; and DBA-IT-RE-3 records XZ Utils as an adversarial event found before use, its actor unidentified. The instruments make no finding of fault. This revision takes in their errata at Sections 2.4, 3.2, 3.3, 4.6, 5.5, 6.2 and 8.1 and their recommendations at Sections 3.3, 4.3, 4.4, 4.6, 5.2, 5.5 and 5.6. The use case for the governing scenario of Section 6.3 is DBA-IT-UC-1.

10.8 The Register Row

The DBA Facility Class Register now carries this class under the framework's name, *Software, Hardware and Product Supply Chain*, as this class asked at v0.1; Project 0 amended the row on 14 October 2026.

10.9 What This Class Notes to Other Instruments

Several elements of this instrument belong to other instruments, and this instrument records them as noted without stating them for those instruments. To the framework: the five parameters of the

blast-radius measure (Section 3.3, Section 10.2), the event categories of Section 5, the three tiers of the sub-index (Section 2.2), the restated geolocation factor of Section 6.1, the declared residuals of Section 6.8, and the conditionality principle of Section 6.9; the conferral section it also asked for is now framework §9.1. To the consuming classes of this project, chemical, care delivery, dispatch, the transportation nodes and control, and the communications operations class: that a vendor's channel is a reachable authority, and that each class's protective functions should be stated as beyond it except through a governed act (Section 4.5, Section 10.5); and that the time to recover a facility's most critical hosts by a path independent of the channel is a figure the facility's continuity should be measured against, stated to the service and not only to the hosts (Section 4.6); and, from *The Faulty Content Update of 19 July 2024* R-1, that a facility whose service cannot wait need not be in a vendor's first ring and may place itself later, which each consuming class should state of its own hosts (Section 4.4). To the electric series, in Project 8: the same two points for the control centers and protective relays it governs. To *The NotPetya Attack on Maersk's Terminals*: that this class cites it for the destructive form of the integrity chain (Section 6.3). The questions it put to Project 0 are decided (Section 10.4) and the register row amended (Section 10.8).

References

DBA-IT Series and Parent Framework

Information Technology Sector Framework (DBA-IT_Sector_Framework). Eclectic Technologies, 2026. Parent sector framework under which this instrument is issued.

DBA-MA Parent Framework. Eclectic Technologies. Source of the design-basis-under-military-action methodology. *Design Basis Layers* (FD-DBL), held for the edition following the October 2026 Edition, is the canonical statement of the four military-action characteristics (§4.4, §6.4).

Defense in Depth (DBA-MA-DiD). Eclectic Technologies, 2026. Declared-residual output form (§3.5).

The NotPetya Attack on Maersk's Terminals (DBA-TRAN-RE-1). Eclectic Technologies, 2026. Held by Project 11; cited by short title and section or finding number.

The Compromise of the SolarWinds Orion Platform (DBA-IT-RE-1), *The Faulty Content Update of 19 July 2024* (DBA-IT-RE-2) and *The Backdoor in the XZ Utils Compression Library* (DBA-IT-RE-3). Eclectic Technologies, 2026. Held by Project 11; conferred at Information Technology Sector Framework §9.1 and cited by short title and finding or recommendation number.

Chemical Production and Synthesis (DBA-CHEM-PR-FC1), *Care Delivery* (DBA-HPH-CD-FC1), *Dam Structures and Water-Control Works* (DBA-DAMS-DS-FC1), *Nodes and Terminals* (DBA-TRAN-ND-FC1) and *Public-Safety Answering and Dispatch* (DBA-ESS-AD-FC1). Eclectic Technologies, 2026. The structural model for this instrument and the consuming classes it notes.

Facility-Class Types Are Physical, Coordination and Logical, author's determination of 30 September 2026, and *A Protective Function Running Logic Other Than What Was Tested Is Not Covered by the Carve-Out*, author's determination of 8 October 2026, both recorded by Project 0. *Closing the Seven Sectors*, working plan of 13 October 2026, Project 9. *Three Author Decisions: Continuity Classes, Vendor Releases and Staged Releases*, author's determination of 14 October 2026, recorded by Project 0.

Foundational Definitions and Command Broker Family

FD-BL, FD-BR, FD-LD, FD-LS, FD-EV, FD-BL-D1 and FD-BL-D2, cited by designation; version state in FD-REG (*Governing Stack Register*). CB-Framework_Cross_Sector (*Command Broker Framework*), cited by designation; version state in CB-REG (*Command Broker Family Register*).

Regulatory, Standards and Guidance

Executive Order 14028, *Improving the Nation's Cybersecurity*, 12 May 2021. National Institute of Standards and Technology. *Secure Software Development Framework (SSDF) Version 1.1*, Special Publication 800-218, February 2022. Office of Management and Budget. Memorandum M-22-18, *Enhancing the Security of the Software Supply Chain through Secure Software Development Practices*, September 2022. Cybersecurity and Infrastructure Security Agency. *Secure Software Development Attestation Form*, 2024; *Secure by Design* program and pledge, 2023 and 2024.

National Telecommunications and Information Administration. *The Minimum Elements for a Software Bill of Materials (SBOM)*, July 2021. Open Source Security Foundation. *Supply-chain Levels for Software Artifacts (SLSA)* specification. International Electrotechnical Commission. IEC 62443-4-1, *Security for industrial automation and control systems — Part 4-1: Secure product development lifecycle requirements*. Regulation (EU) 2024/2847 of the European Parliament and of the Council (Cyber Resilience Act).

Department of Commerce, Bureau of Industry and Security. Final determination prohibiting transactions involving the information and communications technology and services of Kaspersky Lab, 20 June 2024. Cybersecurity and Infrastructure Security Agency and partners. Advisory AA24-038A, *PRC State-Sponsored Actors Compromise and Maintain Persistent Access to U.S. Critical Infrastructure*, 7 February 2024.

Events and Investigations

SolarWinds Corporation. Form 8-K, 14 December 2020; *New Findings From Our Investigation of SUNBURST*, January 2021. Cybersecurity and Infrastructure Security Agency. Emergency Directive 21-01, *Mitigate SolarWinds Orion Code Compromise*, 13 December 2020, and supplemental guidance. National Counterintelligence and Security Center. *SolarWinds Orion Software Supply Chain Attack*, 2021. Government Accountability Office. *Cybersecurity: Federal Response to SolarWinds and Microsoft Exchange Incidents*, GAO-22-104746, January 2022.

CrowdStrike. *Preliminary Post Incident Review*, July 2024; *Channel File 291 Incident Root Cause Analysis*, 6 August 2024; remediation and guidance hub. Microsoft. Statement on the number of Windows devices affected, 20 July 2024. House Committee on Homeland Security, Subcommittee on Cybersecurity and Infrastructure Protection. Hearing, *An Outage Strikes: Assessing the Global Impact of CrowdStrike's Faulty Software Update*, 24 September 2024, Serial No. 118-81.

National Vulnerability Database. CVE-2024-3094. Cybersecurity and Infrastructure Security Agency. Alert, *Reported Supply Chain Compromise Affecting XZ Utils Data Compression Library, CVE-2024-3094*, 29 March 2024. The oss-security mailing list report of 29 March 2024 and the advisories of the affected distributions, among them openSUSE's of 29 March 2024.

Reuters. Rogue communication devices found in Chinese solar power inverters, 14 May 2025, as reported; a candidate for the hardware revision of Section 10.3 and not relied on in this instrument.

Cybersecurity and Infrastructure Security Agency. Emergency Directive 22-02, *Mitigate Apache Log4j Vulnerability*, December 2021. Kaseya. Statements on the VSA incident, July 2021. Congressional Research Service. *CrowdStrike IT Outage: Impacts to Public Safety Systems and Considerations for Congress*, IF12717, version of 4 December 2024.

Revision History

Version	Date	Author	Description
0.1 — DRAFT	October 2026	Tim Roxey	Initial issue. First facility-class instrument in the DBA-IT series and the first logical class written in the corpus, typed logical under the author's determination of 30 September 2026, in the ten-section structure of DBA-HPH-CD-FC1 and DBA-CHEM-PR-FC1 adapted for a class without a site. Written under the plan Closing the Seven Sectors on the author's decisions of 13 October 2026: Software, Hardware and Product Supply Chain as the sector's first class, ahead of Connectivity and Protocols; software in full and hardware and devices at sub-index depth; three reference events proposed, SolarWinds, the faulty content update of July 2024 and XZ Utils, each checked against primary sources, with NotPetya cited as DBA-TRAN-RE-1; and the class's reading of one action for a release stated and referred to Project 0 rather than settled. Proposes the five-parameter blast-radius measure the framework leaves open, the update channel as the governing pathway, and the Bounded Propagation Condition. Checked before issue by an independent review agent, not a person, against primary sources and for internal consistency; its fourteen findings were taken in, among them the Kaseya case restated as abuse of the channel rather than insertion into the build, and the XZ Utils case placed on the edge of the first declared residual.

Version	Date	Author	Description
0.2 — DRAFT	October 2026	Tim Roxey	Conforms to the framework at v0.5 and takes in the return of Project 11's Information Technology batch and the author's determination of 14 October 2026 on vendor releases and staged releases. The three reference events proposed at v0.1 are stated as conferred, as DBA-IT-RE-1 (SolarWinds), DBA-IT-RE-2 (the content update of 19 July 2024) and DBA-IT-RE-3 (XZ Utils), at Sections 1.3, 5.1, 5.5, 5.6, 9.2 and 10.7. Erratum 1 dates dwell from January 2019 as well as September 2019, each attributed, bounds the campaign through this product rather than the whole, and adds the signing period (Sections 3.2, 3.3, 5.5, 6.2). Erratum 2 says many hosts, not all, could not take the correction, and states the 911 consequence from the Congressional Research Service's IF12717 with the hearing's statements attributed (Sections 2.4, 3.2, 4.6, 5.1, 8.1). The ten recommendations enter at Sections 3.3 (held credentials, declared behavior, recovery to the service), 4.3 (readable source, reproducible releases, an independent build in fact), 4.4 (facilities that cannot wait out of the first ring), 4.6 (lowest privilege; credited recovery paths), 5.2 (the exposed population from a record), 5.5 and 5.6 (the pace of intake; a change of maintainer as a change to the product). On the determination, the vendor's controls are carried by contract (Section 4.1), the vendor-side reading of Section 7.3 is restated and Section 10.4 closes; the staged-release reading stands as proposed. Section 6.2 records the ruling that a state's surveillance is military action. Sections 10.8 and 10.9 record the register row and conferral section as made, and note the first-ring recommendation to the consuming classes. The cover lineage stays as issued. No change to the class's scope, chains, parameters, event categories or residuals; the v0.1 entry is left as a record.